Chapter 6 — Solving the linear system

Conditioning of finite element matrices, direct and iterative solvers

MTH3340 · Numerical Methods for PDEs

1 Where the cost is

Every chapter so far ended in the same place:

$$A\vec{\boldsymbol{u}} = \vec{\boldsymbol{f}}, \qquad A \in \mathbb{R}^{n\times n}, \quad \vec{\boldsymbol{f}} \in \mathbb{R}^n, \qquad u_h(\boldsymbol{x}) = \sum_{a=1}^{n} u^a \phi^a(\boldsymbol{x}).$$

Notation: $\vec{\boldsymbol{u}} = (u^1, \ldots, u^n)^T$ is the vector of DOFs (arrow), u_h the function.

- A is square and, for a well-posed problem, non-singular: unique solution;
- u_h does not depend on the basis of V_h; A, $\vec{\boldsymbol{f}}$, $\vec{\boldsymbol{u}}$ do;
- n grows with refinement: $n \sim h^{-d}$. For $h = 10^{-2}$: 10^2 unknowns in 1D, 10^4 in 2D, 10^6 in 3D.

In a finite element code, the solution of $A\vec{\boldsymbol{u}} = \vec{\boldsymbol{f}}$ dominates the computational cost. Today: (1) how *hard* are finite element matrices? — conditioning, §6.3; (2) solve exactly: direct methods, §6.4; (3) solve approximately: iterative methods and preconditioning, §6.5.

2 Eigenvalues and the condition number

$A\vec{u} = \lambda\vec{u}, \vec{u} \neq 0$. Spectrum $\sigma(A) \subset \mathbb{C}$; spectral radius $\rho(A) \doteq \max_{\lambda\in\sigma(A)} |\lambda|$; kernel $\mathrm{Ker}(A) \doteq \{\vec{u} : A\vec{u} = 0\}$ (non-trivial iff A singular).

Induced matrix norm and condition number, for a vector norm $\|\cdot\|_*$ (e.g. $\|\vec{u}\|_2$, or $\|\vec{u}\|_A \doteq \sqrt{\vec{u}^T A\vec{u}}$ for A s.p.d.):

$$\|B\|_* \doteq \max_{\vec{u}\neq 0} \frac{\|B\vec{u}\|_*}{\|\vec{u}\|_*}, \qquad \kappa_*(A) \doteq \|A\|_*\|A^{-1}\|_* \geq 1.$$

$\kappa = \infty$ if A is singular. κ measures how *hard* the system is: it controls the amplification of rounding errors and, today, the speed of iterative convergence.

Q: What is $\kappa(A)$ for $A = c\,\mathbf{I}$, $c \in \mathbb{R}$, $c \neq 0$?

3 The condition number of an s.p.d. matrix

A symmetric positive definite ($\vec{\boldsymbol{u}}^T A\vec{\boldsymbol{u}} > 0$ for $\vec{\boldsymbol{u}} \neq 0$): real positive eigenvalues $\lambda_{\min} \leq \cdots \leq \lambda_{\max}$ and an orthonormal eigenbasis $\{\vec{\boldsymbol{v}}_i\}$, $\vec{\boldsymbol{v}}_i^T \vec{\boldsymbol{v}}_j = \delta_{ij}$.

Expand $\vec{\boldsymbol{u}} = \sum_i \alpha_i \vec{\boldsymbol{v}}_i$, so that $A\vec{\boldsymbol{u}} = \sum_i \alpha_i \lambda_i \vec{\boldsymbol{v}}_i$ and $\|\vec{\boldsymbol{u}}\|_2^2 = \sum_i \alpha_i^2$:

$$\|A\|_2 = \max_{\vec{\boldsymbol{u}} \neq 0} \frac{\|\sum_i \alpha_i \lambda_i \vec{\boldsymbol{v}}_i\|_2}{\|\sum_i \alpha_i \vec{\boldsymbol{v}}_i\|_2} = \max_{\alpha \neq 0} \sqrt{\frac{\sum_i \alpha_i^2 \lambda_i^2}{\sum_i \alpha_i^2}} = \lambda_{\max},$$

$$A^{-1}\vec{\boldsymbol{v}}_i = \tfrac{1}{\lambda_i}\vec{\boldsymbol{v}}_i \;\Rightarrow\; \|A^{-1}\|_2 = \frac{1}{\lambda_{\min}}.$$

Hence

$$\kappa_2(A) = \frac{\lambda_{\max}}{\lambda_{\min}} :$$

well conditioned if $\kappa_2 \approx 1$; ill conditioned as $\lambda_{\min} \to 0$ or $\lambda_{\max} \to \infty$.

4 Rayleigh quotients

Courant–Fischer (min-max): for A symmetric, the extreme eigenvalues are extreme values of the Rayleigh quotient

$$R_A(\vec{u}) \doteq \frac{(A\vec{u},\vec{u})}{(\vec{u},\vec{u})}, \qquad \lambda_{\max} = \max_{\vec{u}\neq 0} R_A(\vec{u}), \qquad \lambda_{\min} = \min_{\vec{u}\neq 0} R_A(\vec{u}).$$

So: bounding $\vec{u}^T A\vec{u}$ above and below by $c\,\vec{u}^T\vec{u}$ bounds the spectrum.

Lemma (Generalised eigenvalue problems — notes, Lemma 6.2.1)

Let A, B be s.p.d. The n eigenvalues of

$$A\vec{u} = \lambda B\vec{u}, \qquad B^{-1}A\vec{u} = \lambda\vec{u}, \qquad B^{-\frac{1}{2}}AB^{-\frac{1}{2}}\vec{u} = \lambda\vec{u}$$

coincide; hence $\kappa(B^{-1}A) = \kappa(B^{-\frac{1}{2}}AB^{-\frac{1}{2}})$, and

$$\lambda_{\min} = \min_{\vec{u}\neq 0} \frac{\vec{u}^T A\vec{u}}{\vec{u}^T B\vec{u}}, \qquad \lambda_{\max} = \max_{\vec{u}\neq 0} \frac{\vec{u}^T A\vec{u}}{\vec{u}^T B\vec{u}}.$$

5 Finite element matrices are inner products

Reaction and Laplacian terms on $V_h = \operatorname{span}\{\phi^1, \ldots, \phi^n\}$:

$$M_{ab} \doteq \int_\Omega \phi^a \phi^b, \qquad L_{ab} \doteq \int_\Omega \nabla\phi^a \cdot \nabla\phi^b, \qquad a, b \in \{1, \ldots, n\}.$$

With $u_h = \sum_a \phi^a u^a$ and $v_h = \sum_b \phi^b v^b$, and taking $\vec{v} = \vec{u}$ on the right:

$$\vec{u}^T M \vec{v} = \int_\Omega u_h v_h, \qquad \vec{u}^T M \vec{u} = \|u_h\|_{L^2}^2,$$

$$\vec{u}^T L \vec{v} = \int_\Omega \nabla u_h \cdot \nabla v_h, \qquad \vec{u}^T L \vec{u} = |u_h|_{H^1}^2,$$

$$\vec{u}^T (M + L)\, \vec{v} = \int_\Omega \left(u_h v_h + \nabla u_h \cdot \nabla v_h\right), \qquad \vec{u}^T (M + L)\, \vec{u} = \|u_h\|_{H^1}^2 :$$

the L^2, H_0^1 and H^1 inner products, and the corresponding squared norms.

The Rayleigh quotients of M and L thus compare $\|u_h\|_{L^2}$ and $|u_h|_{H^1}$ with the *Euclidean* norm of the DOF vector: bounding them is a statement about finite element functions.

6 The mass matrix is well conditioned

Theorem (Spectral bounds for the mass matrix)

On a shape-regular mesh of size h in $\mathbb{R}^d$,

$$c\,h^d \;\le\; \frac{\langle M\vec{v},\vec{v}\rangle}{\langle \vec{v},\vec{v}\rangle} \;\le\; C\,h^d \qquad \forall \vec{v}\in\mathbb{R}^n\setminus\{0\},$$

i.e. $\lambda_{\min}(M)$ and $\lambda_{\max}(M)$ are both $\sim h^d$.

Split into cells and pull back to $\hat{K}$; $gl(a,K)$ is the global-to-local index:

$$\int_\Omega u_h^2 = \sum_{a,b} u^a u^b \sum_{K\in\mathcal{T}_h} \int_{\hat{K}} \hat{\phi}^{gl(a,K)}\hat{\phi}^{gl(b,K)}\,|J_K|\,\mathrm{d}\hat{K}, \qquad |J_K| = Ch^d.$$

What is left is the reference mass matrix $\hat{M}_{ab} = \int_{\hat{K}} \hat{\phi}^a\hat{\phi}^b$, $a,b\le\hat{n}$: s.p.d., independent of h. Each node has a bounded number of neighbours, so

$$Ch^d\lambda_{\min}(\hat{M})\,\langle\vec{u},\vec{u}\rangle \;\le\; \int_\Omega u_h^2 \;\le\; Ch^d\lambda_{\max}(\hat{M})\,\langle\vec{u},\vec{u}\rangle. \qquad \square$$

$\kappa(M) = \mathcal{O}(1)$: refining the mesh does not hurt.

7 The stiffness matrix is not

Theorem (Spectral bounds for the Laplacian matrix)

$$c\,h^d \;\le\; \frac{\langle L\vec{v},\vec{v}\rangle}{\langle \vec{v},\vec{v}\rangle} \;\le\; C\,h^{d-2} \qquad \forall \vec{v}\in\mathbb{R}^n\setminus\{0\},$$

i.e. $\lambda_{\min}(L) \ge ch^d$ *and* $\lambda_{\max}(L) \le Ch^{d-2}$.

Upper: the inverse inequality (Week 8), then the mass bound,

$$\langle L\vec{u},\vec{u}\rangle = \|\nabla u_h\|^2 \le Ch^{-2}\|u_h\|^2 \le Ch^{d-2}\langle \vec{u},\vec{u}\rangle;$$

lower: the Poincaré inequality, then the mass bound,

$$\langle L\vec{u},\vec{u}\rangle = \|\nabla u_h\|^2 \ge C\|u_h\|^2 \ge Ch^{d}\langle \vec{u},\vec{u}\rangle. \qquad \square$$

$\kappa(L) = O(h^{-2})$: halve h, four times worse. Refinement makes the system *harder*, not only bigger.

8 The 1D numbers

Linear elements on a uniform mesh of $(0,1)$, $h = 1/(n+1)$; both matrices are tridiagonal and their eigenvalues are known in closed form.

Mass: $M = \frac{h}{6}\,\mathrm{tridiag}(1,4,1)$, eigenvalues $\frac{h}{3}\big(2+\cos\frac{k\pi}{n+1}\big) \in (\frac{h}{3}, h)$, so $\kappa(M) < 3$ for every h.

Stiffness: $L = \frac{1}{h}\,\mathrm{tridiag}(-1,2,-1)$, eigenvalues $\lambda_k = \frac{2}{h}\big(1-\cos(k\pi h)\big)$, so $\lambda_{\min} \approx \pi^2 h$ and $\lambda_{\max} \approx 4/h$: $\kappa(L) \approx 4/(\pi^2 h^2)$.

h	$\lambda_{\min}$	$\lambda_{\max}$	$\kappa(L)$
10^{-1}	0.98	39	40
10^{-2}	0.099	400	$4.1 \cdot 10^3$
10^{-3}	0.0099	4000	$4.1 \cdot 10^5$

9 Direct methods: Gaussian elimination

Robust, for *any* non-singular matrix. Eliminate column by column, $A \doteq A^{(0)}$:

$$\begin{bmatrix} a_{11}^{(0)} & a_{12}^{(0)} & \cdots & a_{1n}^{(0)} \\ a_{21}^{(0)} & a_{22}^{(0)} & \cdots & a_{2n}^{(0)} \\ \vdots & \vdots & & \vdots \\ a_{n1}^{(0)} & a_{n2}^{(0)} & \cdots & a_{nn}^{(0)} \end{bmatrix} \to \begin{bmatrix} a_{11}^{(0)} & a_{12}^{(0)} & \cdots & a_{1n}^{(0)} \\ 0 & a_{22}^{(1)} & \cdots & a_{2n}^{(1)} \\ \vdots & \vdots & & \vdots \\ 0 & a_{n2}^{(1)} & \cdots & a_{nn}^{(1)} \end{bmatrix} \to \cdots \to \begin{bmatrix} a_{11}^{(0)} & a_{12}^{(0)} & \cdots & a_{1n}^{(0)} \\ 0 & a_{22}^{(1)} & \cdots & a_{2n}^{(1)} \\ \vdots & & \ddots & \vdots \\ 0 & 0 & \cdots & a_{nn}^{(n-1)} \end{bmatrix} = U.$$

Step 1 replaces row i by $A(i,:) - l_{i1}\,A(1,:)$ with the multipliers $l_{i1} \doteq a_{i1}^{(0)}/a_{11}^{(0)}$, $i = 2, \ldots, n$. In matrix form, $A^{(1)} = L_{(1)}^{-1} A$:

$$L_{(1)}^{-1} = \begin{bmatrix} 1 & 0 & \cdots & 0 \\ -l_{21} & 1 & & \\ \vdots & & \ddots & \\ -l_{n1} & 0 & & 1 \end{bmatrix}, \qquad L_{(1)} = \begin{bmatrix} 1 & 0 & \cdots & 0 \\ l_{21} & 1 & & \\ \vdots & & \ddots & \\ l_{n1} & 0 & & 1 \end{bmatrix}$$

unit lower triangular.

10 Factorise once, substitute twice: $A = LU$

After $n-1$ steps, $U = L_{(n-1)}^{-1} \cdots L_{(1)}^{-1} A$, so

$$A = LU, \qquad L = L_{(1)} \cdots L_{(n-1)}, \qquad U = L_{(n-1)}^{-1} \cdots L_{(1)}^{-1} A,$$

with L unit lower triangular (it stores the multipliers) and U upper triangular. The system is then solved by two triangular solves:

$$A\vec{x} = \vec{f} \longrightarrow L(U\vec{x}) = \vec{f} \longrightarrow \underbrace{L\vec{y} = \vec{f}}_{\text{forward substitution}}, \qquad \underbrace{U\vec{x} = \vec{y}}_{\text{backward substitution}}.$$

- the substitutions are cheap, $O(n^2)$ for a dense matrix, against $O(n^3)$ for the elimination: the cost is computing L and U;
- the factorisation is done once: many right-hand sides come almost for free;
- for A s.p.d. one can take $U = L^T$ (Cholesky), half the work and memory.

What a direct solver costs

	flops	memory
dense $n \times n$	$\mathcal{O}(n^3)$	$\mathcal{O}(n^2)$
sparse FE matrix, stored	—	$\mathcal{O}(n \cdot nnz)$
sparse FE matrix, 3D, factorised (nested dissection)	$\mathcal{O}(n^2)$	$\mathcal{O}(n^{4/3})$

- FE matrices are sparse: nnz non-zeros per row, set by d and the order p (the local DOFs), not by n;
- but elimination creates fill-in: L and U are far less sparse than A. Orderings (nested dissection) limit it, up to the bounds above;
- hard to parallelise: not scalable beyond $\mathcal{O}(100)$ processors on distributed memory.

In summary: the method of choice for small and medium problems, in 1D and 2D; not viable for large-scale 3D simulations. There: iterative solvers with robust preconditioners.

11 Iterative solvers and preconditioners

Approximate $\vec{u}$ up to a tolerance. The best (multigrid, domain decomposition) reach linear complexity: $O(n)$ flops and memory, i.e. a bounded number of $O(n)$ iterations; beyond this unit. We study their *building blocks*.

Definition (Preconditioner)

$B = A + E$, E the perturbation. A good B: B^{-1} cheap to apply (and parallel), and $B^{-1}A$ close to I ($\kappa(B^{-1}A) \approx 1$).

The system is equivalent to a fixed-point problem,

$$A\vec{u} = \vec{f} \quad \Longleftrightarrow \quad B\vec{u} = \vec{f} + E\vec{u},$$

which suggests the iteration $B\vec{u}^{k+1} = \vec{f} + E\vec{u}^{k}$: one solve with B per step, instead of one solve with A.

12 The Richardson method

Definition (Preconditioned Richardson method)

Given $\vec{u}^0$, for $k = 0, 1, \ldots$ compute the residual and update:

$$\vec{r}^k \doteq \vec{f} - A\vec{u}^k, \qquad \vec{u}^{k+1} = \vec{u}^k + B^{-1}\vec{r}^k.$$

$B = I$: Richardson method; otherwise preconditioned Richardson method.

The two forms agree: $B\vec{u}^{k+1} = \vec{f} - A\vec{u}^k + B\vec{u}^k = \vec{r}^k + B\vec{u}^k$.

$\vec{r}^k = 0$ means convergence: then $A\vec{u}^k = \vec{f}$, so $\vec{u}^k = \vec{u}$ since A is non-singular. In practice we never get there: the user must provide a tolerance tol and stop when $\|\vec{r}^k\| \le \text{tol}\,\|\vec{f}\|$ (relative residual).

Relaxation. Take only part of the correction,

$$\vec{u}^{k+1} = \vec{u}^k + \alpha\, B^{-1}(\vec{f} - A\vec{u}^k), \qquad \alpha > 0 \qquad \text{(notes, eq. (6.1))}.$$

A small α is more stable but slower; the choice of α is studied next.

13 Error of the Richardson method

Let $\vec{e}^k \doteq \vec{u} - \vec{u}^k$. Subtract $\vec{u}$ from (6.1) and use $\vec{f} = A\vec{u}$:

$$\vec{e}^{k+1} = \vec{e}^k - \alpha B^{-1} A \vec{e}^k = (I - \alpha B^{-1} A)\, \vec{e}^k \doteq G\, \vec{e}^k.$$

The error is propagated by the iteration matrix G: $\vec{e}^k = G^k \vec{e}^0$.

Theorem (Convergence of the Richardson method)

Let A, B be s.p.d., let $\lambda_i \doteq \lambda_i(B^{-1}A)$ (real and positive by Lemma 6.2.1) and $\|\vec{v}\|_B \doteq \sqrt{\vec{v}^T B \vec{v}}$. Then

$$\|\vec{e}^{k+1}\|_B \le \rho(G)\, \|\vec{e}^k\|_B,$$

$$\rho(G) = \max_i |1 - \alpha\lambda_i| = \max\left(|1 - \alpha\lambda_{\min}|,\ |1 - \alpha\lambda_{\max}|\right),$$

hence $\|\vec{e}^k\|_B \le \rho(G)^k \|\vec{e}^0\|_B$, and the method converges for every $\vec{u}^0$ if $\rho(G) < 1$.

Proof of the convergence theorem

The eigenvalues of $G = I - \alpha B^{-1}A$ are $1 - \alpha\lambda_i$, so $\rho(G) = \max_i |1 - \alpha\lambda_i|$; the two extreme values give the maximum since $\alpha > 0$.

G is not symmetric, so $\|G\|_2 \neq \rho(G)$ in general (it can even be > 1 while $\rho(G) < 1$). This is why we measure the error in the B-norm, $\|\vec{v}\|_B = \|B^{1/2}\vec{v}\|_2$:

$$\|G\|_B = \max_{\vec{v}\neq 0} \frac{\|B^{1/2}G\vec{v}\|_2}{\|B^{1/2}\vec{v}\|_2} = \|B^{1/2}GB^{-1/2}\|_2 = \|I - \alpha B^{-1/2}AB^{-1/2}\|_2 = \max_i |1 - \alpha\lambda_i| = \rho(G),$$

because $I - \alpha B^{-1/2}AB^{-1/2}$ is symmetric (its 2-norm is its spectral radius) and similar to G (same eigenvalues). Hence $\|\vec{e}^{k+1}\|_B = \|G\vec{e}^k\|_B \leq \|G\|_B\|\vec{e}^k\|_B$, and the bound for $\vec{e}^k$ follows by recursion. □

($B = I$: the B-norm is the Euclidean norm and $G = I - \alpha A$ is symmetric; no detour needed.)

14 Choosing the relaxation parameter

From the previous board, as a function of the relaxation parameter,

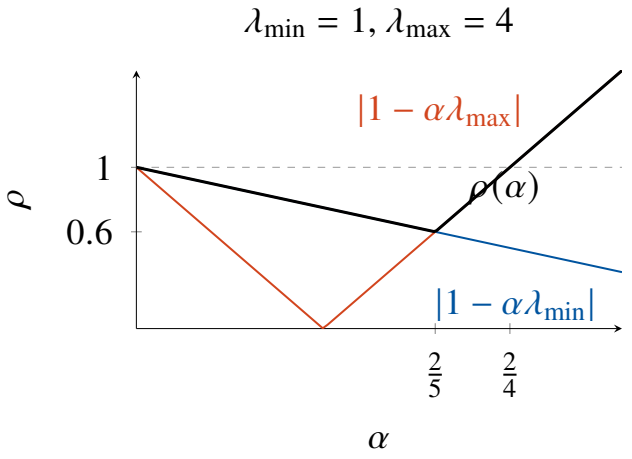
$$\rho(\alpha) = \max\left(|1 - \alpha\lambda_{\min}|,\ |1 - \alpha\lambda_{\max}|\right) :$$

the black curve on the right (blue: $|1 - \alpha\lambda_{\min}|$, orange: $|1 - \alpha\lambda_{\max}|$).
Converges iff $\rho < 1$, i.e. $0 < \alpha < 2/\lambda_{\max}$.

The optimal α balances the two extreme eigenvalues, $1 - \alpha\lambda_{\min} = \alpha\lambda_{\max} - 1$:

$$\alpha_{\text{opt}} = \frac{2}{\lambda_{\max} + \lambda_{\min}},$$

$$\rho_{\text{opt}} = \frac{\lambda_{\max} - \lambda_{\min}}{\lambda_{\max} + \lambda_{\min}} = \frac{\kappa - 1}{\kappa + 1}.$$



But: $\lambda_{\min}$, $\lambda_{\max}$ cost more than the solve itself; without them the method may not converge; with relaxation it is slow; and there is no bound on the number of iterations. Better methods are needed.

15 Steepest descent

For A s.p.d., $\vec{u}$ is the minimiser of the energy

$$\Phi(\vec{x}) \doteq \tfrac{1}{2}\vec{x}^T A\vec{x} - \vec{f}^T\vec{x}, \qquad \text{since} \quad \partial_x\Phi(\vec{x}) = A\vec{x} - \vec{f} = -\vec{r}.$$

Richardson ($B = I$) moves along the residual, i.e. along $-\nabla\Phi$: the direction of steepest descent.

Choose α by minimising Φ on that line (prove it!):

$$\alpha_k = \arg\min_{\alpha\in\mathbb{R}} \Phi(\vec{u}^k + \alpha\vec{r}^k).$$

Definition (Steepest descent)

Given $\vec{u}^0$, for $k = 0, 1, \ldots$:

$$\vec{u}^{k+1} = \vec{u}^k + \alpha_k(\vec{f} - A\vec{u}^k), \qquad \alpha_k = \frac{\langle \vec{r}^k, \vec{r}^k\rangle}{\langle \vec{r}^k, A\vec{r}^k\rangle}.$$

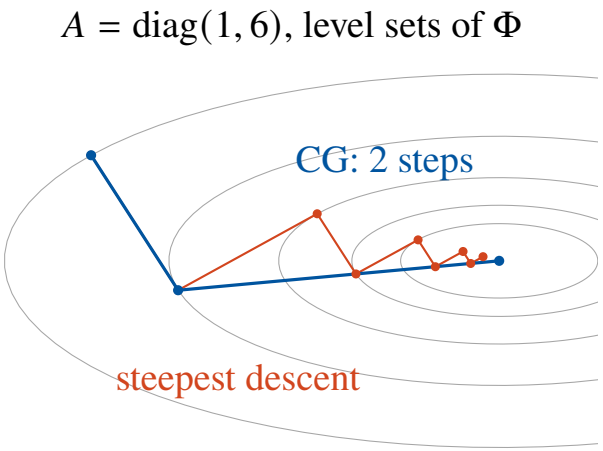
16 Steepest descent in pictures

No spectrum needed, and it always converges:

$$\|\vec{e}^{\,k}\|_A \le \left(\frac{\kappa(A)-1}{\kappa(A)+1}\right)^k \|\vec{e}^{\,0}\|_A .$$

Cost per iteration: a sparse matrix–vector product, $O(n)$, plus two inner products.

Iterations grow with $\kappa(A) \sim h^{-2}$: the total cost deteriorates under refinement. The zig-zag on the right is the reason: each step is orthogonal to the previous one.



17 Conjugate directions

Steepest descent optimises the *distance*, not the *direction*. Suppose we had an A-orthogonal basis of conjugate directions $\{p^k\}_{k=0}^{n-1}$,

$$\langle \vec{\boldsymbol{p}}^{\,i}, A\vec{\boldsymbol{p}}^{\,j}\rangle = 0 \quad (i \neq j), \qquad \vec{\boldsymbol{u}}^{\,k+1} = \vec{\boldsymbol{u}}^{\,k} + \alpha_k \vec{\boldsymbol{p}}^{\,k}, \qquad \alpha_k = \frac{\langle \vec{\boldsymbol{r}}^{\,k}, \vec{\boldsymbol{p}}^{\,k}\rangle}{\langle \vec{\boldsymbol{p}}^{\,k}, A\vec{\boldsymbol{p}}^{\,k}\rangle}.$$

- each α_k minimises Φ along $\vec{\boldsymbol{p}}^{\,k}$, and the directions do not interfere: exact after at most n steps;
- CG was introduced (1952) as a *direct* solver; rounding errors destroy finite termination;
- rediscovered as an *iterative* method: tiny residual after far fewer than n steps. The most used solver in practice for s.p.d. matrices.

18 How to build the conjugate directions

From the residuals, A-orthogonalising against the previous directions (Gram–Schmidt in the A-inner product):

$$\vec{\boldsymbol{p}}^{\,k} = \vec{\boldsymbol{r}}^{\,k} + \sum_{i=0}^{k-1} \beta^i \vec{\boldsymbol{p}}^{\,i}, \qquad \beta^i = -\frac{\langle A\vec{\boldsymbol{r}}^{\,k}, \vec{\boldsymbol{p}}^{\,i}\rangle}{\langle A\vec{\boldsymbol{p}}^{\,i}, \vec{\boldsymbol{p}}^{\,i}\rangle} \quad \text{so that } \langle A\vec{\boldsymbol{p}}^{\,k}, \vec{\boldsymbol{p}}^{\,i}\rangle = 0 \quad \forall i < k.$$

Full orthogonalisation stores all previous directions and costs more at every step.

Key property of CG (exact arithmetic): the residuals are mutually orthogonal, and $\beta^i = 0$ for $i < k-1$; the *previous direction only* is enough,

$$\vec{\boldsymbol{p}}^{\,k} = \vec{\boldsymbol{r}}^{\,k} + \beta_{k-1}\, \vec{\boldsymbol{p}}^{\,k-1}.$$

Fixed cost and memory per iteration, whatever k.

19 The conjugate gradient algorithm

Algorithm (Conjugate gradient — notes, Algorithm 6.1)

$\vec{p}^0 := \vec{r}^0 := \vec{f} - A\vec{x}^0$
for $j = 0, \ldots,$ till CONV **do**
  $\vec{s}^{j+1} = A\vec{p}^j$
  $\alpha_j := (\vec{r}^j, \vec{r}^j)/(\vec{s}^{j+1}, \vec{p}^j)$
  $\vec{x}^{j+1} := \vec{x}^j + \alpha_j \vec{p}^j$
  $\vec{r}^{j+1} := \vec{r}^j - \alpha_j \vec{s}^{j+1}$
  $\beta_j := (\vec{r}^{j+1}, \vec{r}^{j+1})/(\vec{r}^j, \vec{r}^j)$
  $\vec{p}^{j+1} := \vec{r}^{j+1} + \beta_j \vec{p}^j$
end for

Per iteration: one matrix–vector product, two inner products, three updates; three vectors stored.

Only for s.p.d. matrices (Φ must be a quadratic form with a minimum).

Always converges, and $\sqrt{\kappa}$ replaces κ (below): $\sqrt{\kappa(L)} \sim 1/h$, so iterations $\sim 1/h$, not $1/h^2$.

Theorem (Convergence of CG)

$$\|\vec{e}^k\|_A \;\le\; 2\left(\frac{\sqrt{\kappa(A)}-1}{\sqrt{\kappa(A)}+1}\right)^k \|\vec{e}^0\|_A.$$

20 Preconditioned conjugate gradient

We want $\kappa \approx 1$. We cannot change A; we precondition with an s.p.d. B and solve instead

$$B^{-\frac{1}{2}} A B^{-\frac{1}{2}} \vec{v} = B^{-\frac{1}{2}} \vec{f}, \qquad \vec{u} = B^{-\frac{1}{2}} \vec{v}.$$

The matrix is still s.p.d., so CG applies, and by Lemma 6.2.1 its condition number is $\kappa(B^{-1}A)$:

$$\|\vec{e}^{\,k}\|_A \leq 2 \left(\frac{\sqrt{\kappa(B^{-1}A)} - 1}{\sqrt{\kappa(B^{-1}A)} + 1} \right)^k \|\vec{e}^{\,0}\|_A.$$

But $B^{-1/2}$ is costly, and the equivalent system $B^{-1}A\vec{u} = B^{-1}\vec{f}$ is not symmetric.

Rearranging CG (next board): only B^{-1} is ever applied, yet the iterates are exactly those of CG on $B^{-\frac{1}{2}} A B^{-\frac{1}{2}}$.

21 The PCG algorithm

Algorithm (PCG — notes, Algorithm 6.2)

$\vec{r}^0 := \vec{f} - A\vec{x}^0$, $\vec{z}^0 = B^{-1}\vec{r}^0$
$\vec{p}^0 := \vec{z}^0$
for $j = 0, \ldots,$ till CONV **do**
  $\vec{s}^{j+1} = A\vec{p}^j$
  $\alpha_j := (\vec{z}^j, \vec{r}^j)/(\vec{s}^{j+1}, \vec{p}^j)$
  $\vec{x}^{j+1} := \vec{x}^j + \alpha_j \vec{p}^j$
  $\vec{r}^{j+1} := \vec{r}^j - \alpha_j \vec{s}^{j+1}$
  $\vec{z}^{j+1} := B^{-1}\vec{r}^{j+1}$
  $\beta_j := (\vec{z}^{j+1}, \vec{r}^{j+1})/(\vec{z}^j, \vec{r}^j)$
  $\vec{p}^{j+1} := \vec{z}^{j+1} + \beta_j \vec{p}^j$
end for

Compared with CG: one extra vector $\vec{z}$ and, per iteration, one application of B^{-1} next to the product with A.

$B = I$ gives back CG. $B = \mathrm{diag}(A)$ (Jacobi) is the cheapest non-trivial choice.

The open problem: a cheap, robust B for finite element matrices, with $\kappa(B^{-1}A)$ bounded independently of h — multigrid, domain decomposition (not in this unit).

22 Stopping criteria and tolerances

An iterative solver returns an *approximation*; the user decides when it is good enough. The computable quantity is the residual $\vec{r}^{\,k} = \vec{f} - A\vec{u}^{\,k}$ (the error $\vec{e}^{\,k}$ is unknown).

- absolute: $\|\vec{r}^{\,k}\| \le \mathrm{tol}_{\mathrm{abs}}$ — depends on the scaling of the equations (multiply A and $\vec{f}$ by 10^6 and the same iterate fails the test);
- relative: $\|\vec{r}^{\,k}\| \le \mathrm{tol}_{\mathrm{rel}}\,\|\vec{f}\|$, scale-invariant, the usual choice; or relative to the initial residual, $\|\vec{r}^{\,k}\| \le \mathrm{tol}_{\mathrm{rel}}\,\|\vec{r}^{\,0}\|$, which depends on $\vec{u}^{\,0}$;
- in practice both: $\|\vec{r}^{\,k}\| \le \mathrm{tol}_{\mathrm{rel}}\|\vec{f}\| + \mathrm{tol}_{\mathrm{abs}}$, plus a maximum number of iterations as a safeguard.

Residual is not error: $\vec{e}^{\,k} = A^{-1}\vec{r}^{\,k}$, so $\dfrac{\|\vec{e}^{\,k}\|}{\|\vec{u}\|} \le \kappa(A)\,\dfrac{\|\vec{r}^{\,k}\|}{\|\vec{f}\|}$ — a small relative residual guarantees a small relative error only if $\kappa(A)$ is moderate.

How small? In accordance with the discretisation error: solving the linear system much more accurately than $\|u - u_h\|$ is wasted work.

23 Summary

	needs	convergence / cost
LU (direct)	any non-singular A	exact; $O(n^3)$ dense, fill-in if sparse
Richardson	$\alpha < 2/\lambda_{\max}$	$\rho = \max_i \lvert 1 - \alpha\lambda_i \rvert$; needs the spectrum
Steepest descent	A s.p.d.	$\left(\frac{\kappa-1}{\kappa+1}\right)^k$; $O(n)$ per iteration
CG	A s.p.d.	$2\left(\frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}\right)^k$; $\leq n$ steps exactly
PCG	A, B s.p.d.	same with $\kappa(B^{-1}A)$; one B^{-1} per iteration
Multigrid (as B)	elliptic A	$\kappa(B^{-1}A) = O(1)$: $O(n)$ in total

- $\vec{u}^T M \vec{u} = \|u_h\|_{L^2}^2$, $\vec{u}^T L \vec{u} = |u_h|_{H^1}^2$; $\kappa(M) = O(1)$, $\kappa(L) = O(h^{-2})$;
- iterations follow κ (or $\sqrt{\kappa}$): refinement makes the solve harder;
- a preconditioner replaces $\kappa(A)$ by $\kappa(B^{-1}A)$: that is where the research is.

24 Algorithms versus hardware

Computational mathematics has enlarged the problems we can solve by more than the growth of computers has. The standard example, for a 3D finite element system:

sparse direct solver: $O(n^2)$ flops $\longrightarrow$ multigrid: $O(n)$ flops.

- With W flops, elimination solves $n \sim W^{1/2}$ unknowns, multigrid $n \sim W$. A machine that solves $n = 10^3$ by elimination ($W \sim 10^6$) solves $n = 10^6$ with multigrid: a thousand times larger problem, same machine, same energy;
- hardware doubles its speed every ≈ 2 years: a factor 10^3 is about 20 years of new computers, and the energy to run them.

Question. Training a large language model is a non-convex optimisation attacked with Richardson-type methods (stochastic gradient descent, Adam), and its cost has been met with hardware and energy. Is there a "multigrid" for training neural networks? To be discovered.